

CityLearn Auto-UI

1 Abstract

This paper presents an automated visualisation and analysis framework designed to enhance research productivity within the CityLearn 2.5.0 environment. While CityLearn offers an UI, it relies on manual data handling. The proposed system introduces a seamless pipeline that bridges the gap between raw simulation and comparative analysis.

By automating the extraction of building-level time-series data and serving it via a Flask-based API, the framework eliminates manual data handling. The main contribution of this work is a decoupled architecture with a Flask backend and a Plotly frontend which allows for both side-by-side comparisons and multiple metrics within a single interactive plot, enabling researchers to layer performance indicators from different control agents onto a shared temporal axis. The modular dashboard uses blank views where users can add metrics to be compared or analysed, with a coloring logic which allows multi-building data to remain readable by using variations of the same base color.

2 Introduction

The transition towards grid-interactive efficient buildings requires control strategies capable of balancing occupant comfort with grid stability and carbon reduction [1] [2]. CityLearn has emerged as a tool for benchmarking multi-agent reinforcement learning in OpenAI Gymnasium environments [3] [4]. However, a significant bottleneck remains in data analysis.

Standard evaluation practices often rely on aggregate Key Performance Indicators (KPIs) present in static tables. While these provide a good “final score” for evaluating a control strategy, they often hide building-level behaviours which lead to these scores[5]. Moreover, existing visualisation tools require manual data handling, and don't offer side-by-side comparisons which can hinder data analysis or behaviour understanding in simulation runs.

This paper introduces an automated, high-resolution visualisation framework made to be compatible with CityLearn 2.5.0 [3]. This work introduces multiple contributions, first, the automated pipeline eliminates the need for manual data handling by triggering KPI calculations and plot generation immediately after the simulation is complete. Second, a modular dashboard architecture which facilitates side-by-side comparative analysis of multiple simulation runs, and multiple metrics within the same view. Third, the introduction of more granular metrics such as which can provide deeper insight into the simulation. The remainder of this paper is structured as follows;

3 Related Work

The visualisation of and analysis of behaviours in energy management control systems require tools which offer both information and ease of use. This reveals a

space within the existing landscape of simulation interfaces and diagnostic tools, specifically comparing it to the official CityLearn UI and the broader industry standard, TensorBoard.

3.1 The CityLearn Web UI

The official CityLearn WebUI is a dedicated platform designed to visualise standardised simulation outputs. While it offers a suite of utilities, including a custom schema generator, it is primarily made as a post-simulation diagnostic tool. The official interface requires a manual upload of the output folder containing the simulations, which introduces a bottleneck between the simulation execution and data analysis. Although the official UI provides a “Comparison” option for calculating deltas between two simulation runs, these are presented as tabular data rather than synchronised, side-by-side time-series visualisations. While the official tool groups data into categories like consumption, production and storage, our solution offers a modular view that allows users to view multiple building or community level metrics across an unlimited number of panels.

3.2 TensorBoard and General-Purpose RL Logging

TensorBoard is the industry standard for monitoring reinforcement learning training progress, offering highly detailed views of reward curves, loss functions, and weights [6], [7]. While this framework offers great insight into Reinforcement Learning (RL) agent’s performance, it was designed for direct comparison for the same metric between runs or agents, which makes plotting the same metric for multiple buildings (like in the case of CityLearn schemas) not feasible. Unlike TensorBoard which is domain agnostic, our framework is built for the CityLearn environment’s data structure.

The framework described in this paper is not meant as a direct competitor to any of these existing standards, but rather as a complementary tool, as it does not yet implement the “custom schema creation” functionality, nor does it intend to evaluate specific reinforcement learning algorithms performance, but rather help visualise behaviours, and trends within the simulation, which can be used to better understand specific consequences of the decisions taken by the control algorithm.

4 System architecture and Implementation

The core contribution of this work is the creation of a non-intrusive, automated pipeline that synchronizes CityLearn simulations with a visualisation dashboard. A primary design requirement was to maintain compatibility with standard CityLearn workflows without affecting them. To achieve this, the system operates as an external layer that monitors and processes outputs without requiring modifications to the environment or agent logic.

The pipeline begins with normal simulation executions, saving results to the default CityLearn output directory. After a simulation is complete, the system archives the output under a unique identifier and updates a central data registry. This registry allows the Flask API to expose specific runs and KPIs to the frontend dropdown menus. After this, the framework triggers an automated script which iterates through its output folder, and calculates building-level and community-level KPIs. Using the KPIs calculated in the previous step, the framework uses a Flask server to host a single-page application. When a user selects a metric, the frontend fetches the raw data and renders it client-side using [Plotly.js](#). [8]. The framework is designed to provide visibility into individual building behaviours that are often obscured in aggregate views present in existing platforms.

Table x.x shows a list of all calculated KPIs, together with the source of the data used in their calculation and how the data is processed to obtain the KPI.

KPI Name	Source File(s)	Derivation / Calculation Logic
Grid Consumption	exported_data_building_X_ep0.csv	Directly uses 'Net Electricity Consumption-kWh'; Community is the sum across all buildings.
Load	exported_data_building_X_ep0.csv	Uses 'Non-shiftable Load-kWh'; Community is the aggregate sum.
PV Generation	exported_data_building_X_ep0.csv	Uses absolute values of 'Energy Production from PV-kWh' to ensure positive generation values.

Electrical Storage SoC	exported_data_building_X_battery_ep0.csv	Building level uses 'Battery Soc-%'; Community is a weighted average based on individual battery capacities.
Electrical Storage Action	exported_data_building_X_battery_ep0.csv	Uses 'Battery (Dis)Charge-kWh'; positive values represent charging and negative represent discharging.
Electricity Cost	grid_consumption.csv + Pricing File	Multiplies consumption by dynamic pricing from price.csv or community data (defaulting to 0.33 if unavailable).
Carbon Emissions	grid_consumption.csv + Community	Multiplies consumption by 'Carbon Intensity-kg_CO2/kWh' from community files.

Total Cost	total_cost.csv	Cumulative sum of all community-level costs over the entire simulation period.
Total Carbon Emissions	total_carbon_emissions.csv	Cumulative sum of all community-level carbon emissions.
Max Consumption	total_grid_consumption.csv	Identifies the single maximum value in the community grid consumption time-series.
Max Load	total_load.csv	Identifies the maximum value in the community non-shiftable load time-series.
Total PV Generation	total_pv_generation.csv	Total electricity generated by all community photovoltaic panels over the simulation duration.

Average Storage SoC	total_electrical_storage_soc.csv	Mean of the weighted average community state of charge over time.
Building Charge/Discharge	electrical_storage_action.csv	Aggregates individual building charging and discharging energy (sums positive/negative values separately).
Total Charge/Discharge	electrical_storage_action.csv	Community-wide totals for all energy moved into and out of storage systems.

5 Functionality and Interface

The interface functions as a centralized dashboard for analysis. The framework populates a modular template which allows users to visualise multiple building-level or community-level metrics across an unlimited number of panels. A core capability of this interface is the ability to analyse side-by-side time series plots, which is not present in previous frameworks. This type of visualisation can be helpful for direct observation on how different control agents, such as rule-based controllers or reinforcement learning models, react to the same environmental conditions over time, or even how the same control strategy affects different building configurations.

On the UI the user can add as many views as necessary, these are initially blank, and on each view the user can add as many metrics as necessary, by selecting the run ID, the KPI, and a color which will be used for plotting. Users can compare data by adding multiple traces to a single plot or by creating multiple views. Active traces are represented by interactive tokens above the plot, which display the run ID and KPI name.

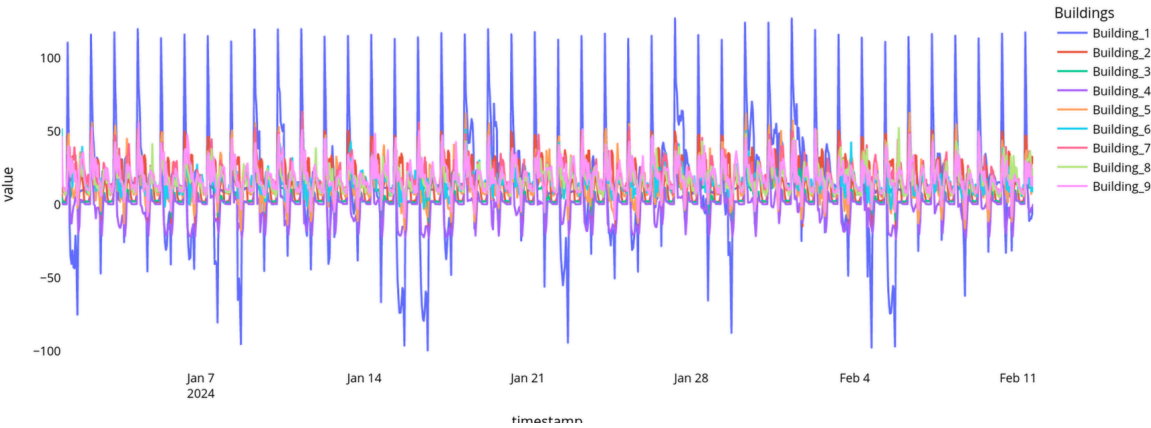
RBC_plld_20260307_233041

Grid Consumption



Grid Consumption

Electricity consumed from the grid by each building over time (in kWh). Negative values indicate electricity exported to the grid.



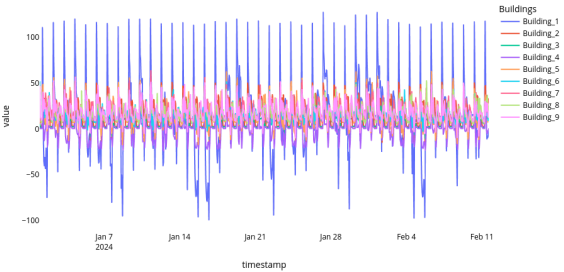
RBC_yhkm_20260307_233757

Grid Consumption



Grid Consumption

Electricity consumed from the grid by each building over time (in kWh). Negative values indicate electricity exported to the grid.



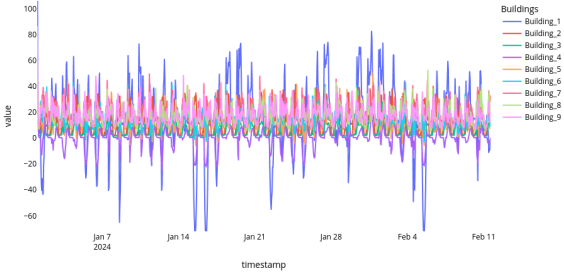
RBC_2_zyfy_20260307_234643

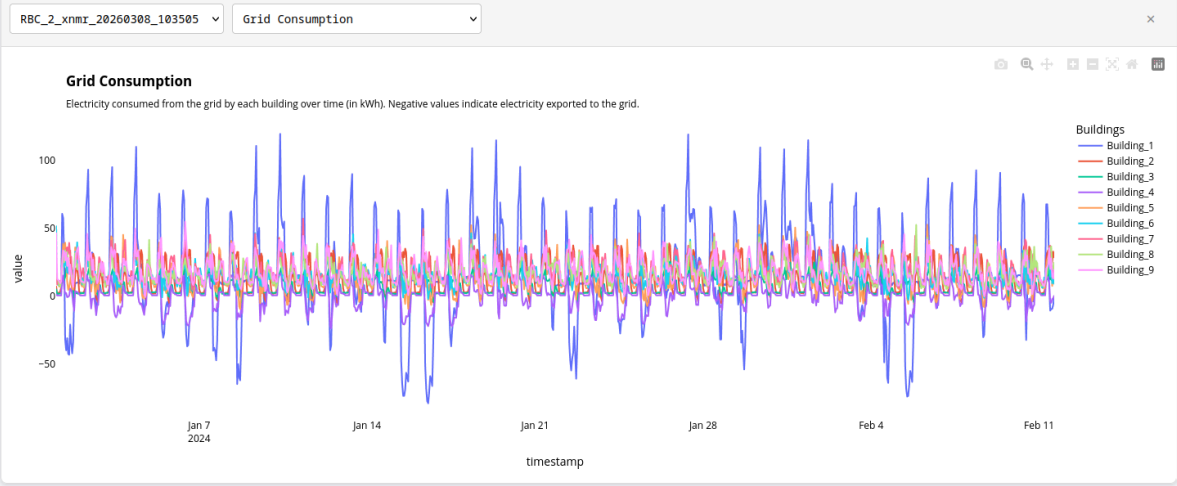
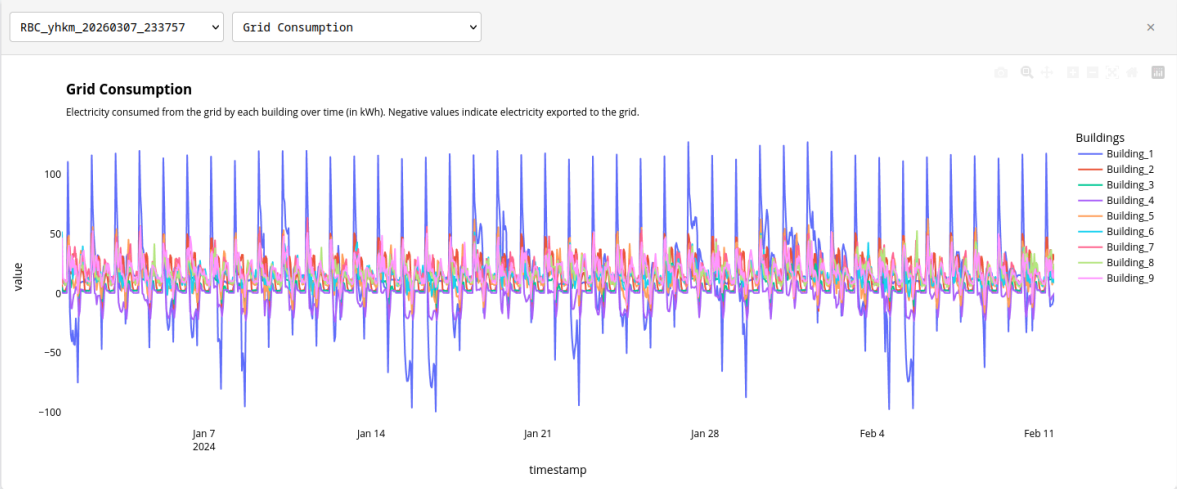
Grid Consumption

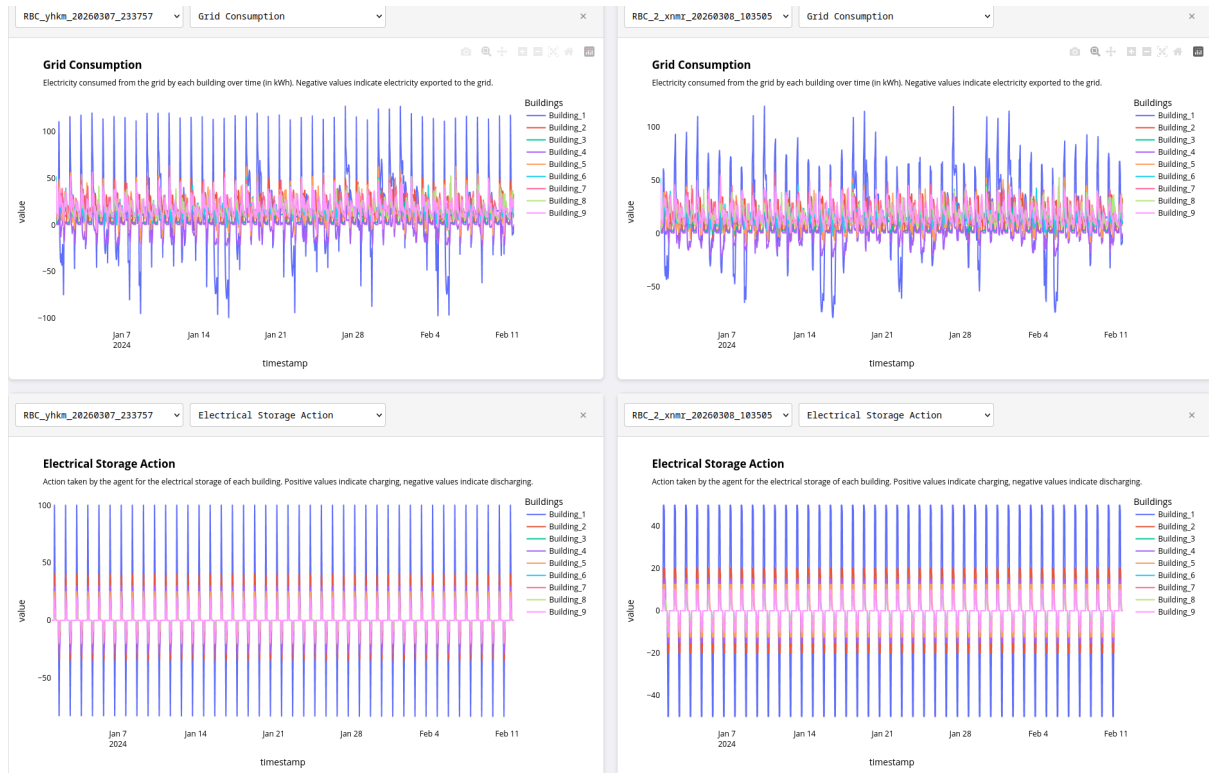


Grid Consumption

Electricity consumed from the grid by each building over time (in kWh). Negative values indicate electricity exported to the grid.







6 Discussion and Future Work

The proposed framework is not intended as a direct replacement for existing evaluation platforms or specialized reinforcement learning monitoring tools like TensorBoard. Instead, it serves as a complementary overview tool focused on behavioral transparency and research productivity. While standard evaluation practices rely on aggregate Key Performance Indicators (KPIs) and static tables, this system prioritizes high-resolution, building-level time-series data. The primary value of the interface lies in its ability to automate the bridge between raw simulation and comparative analysis, removing the latency associated with manual data handling. By offering side-by-side time-series visualizations rather than the tabular deltas found in the official CityLearn WebUI, the framework allows researchers to observe the specific temporal consequences of control algorithm decisions across multiple agents or buildings simultaneously. This approach shifts the focus from a "final score" evaluation to a deeper understanding of trends and behaviors within the simulation environment.

Future development will focus on expanding the system's interactive capabilities and feature parity with existing standards. Future improvements include the integration of more granular UI controls such as time-step interval sliders and data series toggles, which would allow users to isolate specific building components without reloading the dashboard. Incorporating these missing functionalities alongside the existing automated pipeline would create a more robust environment for both district energy coordination and agent development.

This paper has presented CityLearn Auto-UI, an automated visualization and analysis framework designed to enhance research productivity within the CityLearn

2.5.0 environment. By establishing a non-intrusive pipeline that synchronizes simulation outputs with an interactive dashboard, the system effectively removes the bottleneck of manual data processing and the latency associated with manual uploads. The core strength of the platform lies in its ability to provide high-resolution, building-level insights that are frequently obscured by the aggregate scoring typical of standard evaluation practices. Through a modular architecture and dynamic injection mechanism, the framework facilitates synchronized, side-by-side comparisons of diverse control strategies across multiple performance indicators. While not intended to replace established industry standards like TensorBoard or the official CityLearn WebUI, it is meant as a data visualization tool.

Bibliography

- [1] O. Vera-Piazzini, M. Scarpa, and F. Peron, "Building Energy Simulation and Monitoring: A Review of Graphical Data Representation," *Energies*, vol. 16, no. 1, Dec. 2022, doi: 10.3390/en16010390.
- [2] M. Neukomm, V. Nubbe, and R. Fares, "Grid-Interactive Efficient Buildings Technical Report Series: Overview of Research Challenges and Gaps," NREL/TP--5500-75470, DOE/GO--102019-5227, 1577966, Dec. 2019. doi: 10.2172/1577966.
- [3] K. Nweye *et al.*, "CityLearn v2: Energy-flexible, resilient, occupant-centric, and carbon-aware management of grid-interactive communities," *J. Build. Perform. Simul.*, vol. 18, no. 1, pp. 17–38, Jan. 2025, doi: 10.1080/19401493.2024.2418813.
- [4] J. R. Vazquez-Canteli, S. Dey, G. Henze, and Z. Nagy, "CityLearn: Standardizing Research in Multi-Agent Reinforcement Learning for Demand Response and Urban Energy Management," Dec. 18, 2020, *arXiv*: arXiv:2012.10504. doi: 10.48550/arXiv.2012.10504.
- [5] G. Pinto, M. S. Piscitelli, J. R. Vázquez-Canteli, Z. Nagy, and A. Capozzoli, "Coordinated energy management for a cluster of buildings through deep reinforcement learning," *Energy*, vol. 229, p. 120725, Aug. 2021, doi: 10.1016/j.energy.2021.120725.
- [6] M. Abadi *et al.*, "TensorFlow: A system for large-scale machine learning," May 31, 2016, *arXiv*: arXiv:1605.08695. doi: 10.48550/arXiv.1605.08695.
- [7] D. C. Vogelsang and B. J. Erickson, "Magician's Corner: 6. TensorFlow and TensorBoard," *Radiol. Artif. Intell.*, vol. 2, no. 3, p. e200012, May 2020, doi: 10.1148/ryai.2020200012.
- [8] "Interactive Web-Based Data Visualization with R, plotly, and shiny," ResearchGate. Accessed: Mar. 19, 2026. [Online]. Available: https://www.researchgate.net/publication/340147564_Interactive_Web-Based_Data_Visualization_with_R_plotly_and_shiny